GENERATIVE AI

EXPERIMENT-3

Simple GAN with TensorFlow and MNIST Dataset

CODE:

```
# STEP 1–2 : Import Libraries & Load MNIST Dataset

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# Load only images (labels not required for GAN)
(x_train, _), (_, _) = keras.datasets.mnist.load_data()

# STEP 3–5 : Preprocess Images + Define Parameters + DataseT

# Convert to float and normalize to [-1, 1]
x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5

# Add channel dimension (28x28x1)
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)

# Training parameters
batch_size = 64
noise_dim = 100
epochs = 10

# Create shuffled dataset batches
dataset = tf.data.Dataset.from_tensor_slices(x_train)\
                    .shuffle(60000)\
```

```
                    .batch(batch_size)


# STEP 6–7 : Build Generator and Discriminator Models

# -------- Generator --------
generator = keras.Sequential([
    keras.layers.Input(shape=(noise_dim,)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(28 * 28 * 1, activation='tanh'),
    keras.layers.Reshape((28, 28, 1))
])

# -------- Discriminator --------
discriminator = keras.Sequential([
    keras.layers.Input(shape=(28, 28, 1)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])


# STEP 8 : Define Loss Function and Optimizers

loss_fn = keras.losses.BinaryCrossentropy()

g_optimizer = keras.optimizers.Adam(learning_rate=0.0002)
d_optimizer = keras.optimizers.Adam(learning_rate=0.0002)


# STEP 9–12 : GAN Training Loop (Generator + Discriminator)

for epoch in range(epochs):

    for real_images in dataset:
```

```
batch_size_now = real_images.shape[0]

# Labels
real_labels = tf.ones((batch_size_now, 1))
fake_labels = tf.zeros((batch_size_now, 1))

     # Train Discriminator

noise = tf.random.normal((batch_size_now, noise_dim))
fake_images = generator(noise, training=True)

with tf.GradientTape() as tape:

    real_output = discriminator(real_images, training=True)
    fake_output = discriminator(fake_images, training=True)

    real_loss = loss_fn(real_labels, real_output)
    fake_loss = loss_fn(fake_labels, fake_output)

    d_loss = real_loss + fake_loss

d_gradients = tape.gradient(
    d_loss,
    discriminator.trainable_variables
)

d_optimizer.apply_gradients(
    zip(d_gradients, discriminator.trainable_variables)
)

     # Train Generator


noise = tf.random.normal((batch_size_now, noise_dim))
```

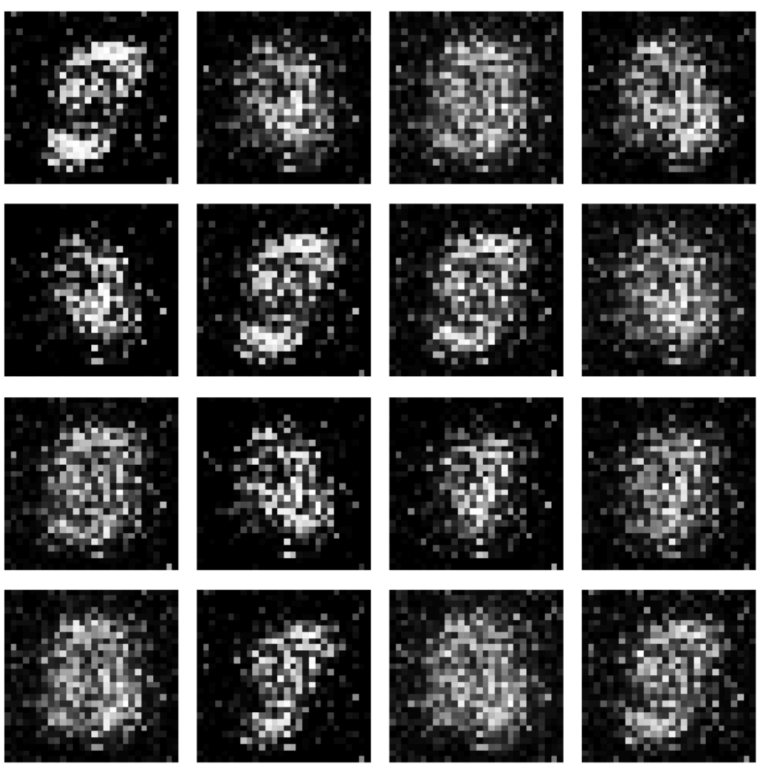
```
        with tf.GradientTape() as tape:

            generated_images = generator(noise, training=True)
            output = discriminator(generated_images, training=True)

            # Generator wants fake images classified as real
            g_loss = loss_fn(real_labels, output)

        g_gradients = tape.gradient(
            g_loss,
            generator.trainable_variables
        )

        g_optimizer.apply_gradients(
            zip(g_gradients, generator.trainable_variables)
        )

    print(f"Epoch [{epoch+1}/{epochs}]  D Loss: {d_loss.numpy():.4f}  G
Loss: {g_loss.numpy():.4f}")


# STEP 13 : Generate Synthetic Images After Training


noise = tf.random.normal((16, noise_dim))
generated_images = generator(noise, training=False)


# STEP 14 : Display Generated Images

fig, axes = plt.subplots(4, 4, figsize=(6, 6))

for i, ax in enumerate(axes.flat):
    ax.imshow((generated_images[i, :, :, 0] + 1) / 2, cmap='gray')
```

```
    ax.axis("off")

plt.tight_layout()
plt.show()


# GAN PERFORMANCE METRICS / CONFIDENCE SCORES


# Take one batch of real images
real_batch = next(iter(dataset))

batch_size_eval = real_batch.shape[0]

# Generate fake images
noise = tf.random.normal((batch_size_eval, noise_dim))
fake_batch = generator(noise, training=False)

# Discriminator predictions
real_pred = discriminator(real_batch, training=False)
fake_pred = discriminator(fake_batch, training=False)


#1 Discriminator Accuracy

real_correct = tf.reduce_sum(tf.cast(real_pred > 0.5, tf.float32))
fake_correct = tf.reduce_sum(tf.cast(fake_pred < 0.5, tf.float32))

d_accuracy = (real_correct + fake_correct) / (2 * batch_size_eval)

print("Discriminator Accuracy:", float(d_accuracy))


#2 Confidence Score (Fake Images Classified as Real)
```

```
confidence_score = tf.reduce_mean(fake_pred)

print("Generator Confidence Score (Fake→Real):",
      float(confidence_score))
```

OUTPUT:

```
Epoch [1/10]  D Loss: 0.4611  G Loss: 1.6366
Epoch [2/10]  D Loss: 0.7118  G Loss: 1.3432
Epoch [3/10]  D Loss: 0.5394  G Loss: 1.7597
Epoch [4/10]  D Loss: 0.7089  G Loss: 1.7305
Epoch [5/10]  D Loss: 0.8450  G Loss: 1.4344
Epoch [6/10]  D Loss: 0.5921  G Loss: 1.8095
Epoch [7/10]  D Loss: 0.6479  G Loss: 1.9139
Epoch [8/10]  D Loss: 0.4668  G Loss: 2.1820
Epoch [9/10]  D Loss: 0.4820  G Loss: 2.3382
Epoch [10/10]  D Loss: 0.3558  G Loss: 3.1734
```



```
Discriminator Accuracy: 0.984375
Generator Confidence Score (Fake → Real): 0.10866151005029678
```

COLLAB LINK → **GenAI EX-3.ipynb**

Question and answers:

1. Why are labels ignored when loading the MNIST dataset?

In a Generative Adversarial Network, the goal is to generate images rather than classify them. Since labels are only needed for classification tasks, they are not required here and are ignored.

2. Why are images normalized to the range [-1, 1]?

The generator uses the tanh activation function, which outputs values between -1 and 1. Matching the input data to this range helps the model learn faster and improves training stability.

3. What is the purpose of setting noise_dim = 100?

The noise vector acts as the input to the generator. It provides randomness, allowing the model to generate diverse images instead of producing the same output every time.

4. How many Dense layers are used in the generator and why?

There are two Dense layers in the generator. These layers help transform random noise into a structured representation that can eventually be reshaped into an image.

5. What is the purpose of the Flatten layer in the discriminator?

The Flatten layer converts a 2D image into a 1D vector so that it can be processed by Dense layers, which only accept one-dimensional input.

6. Why is sigmoid used in the final layer of the discriminator?

Sigmoid outputs values between 0 and 1, which represent probabilities. This makes it suitable for distinguishing between real and fake images.

7. What loss function is used and why?

Binary cross-entropy is used because the discriminator performs a binary classification task, deciding whether an image is real or fake.

8. Why is the discriminator trained on both real and fake images?

Training on both types helps the discriminator learn the difference between authentic images and generated ones, improving its ability to detect fakes.

9. Why does the generator compare its output with real labels?

The generator is trained to fool the discriminator. By using real labels as targets, it learns to produce images that appear real to the discriminator.

10. How many synthetic images are displayed during training?

Sixteen images are displayed in a 4 by 4 grid, which helps in visually evaluating the generator's performance.

11. What happens when the number of epochs is increased to 50?

The generated images become clearer and more recognizable. However, too many epochs can lead to overfitting or mode collapse, where the generator produces similar images repeatedly.

12. What happens if noise_dim is reduced to 50?

Reducing the noise dimension decreases randomness, which can reduce the diversity and quality of the generated images.

13. What is the effect of increasing neurons to 256 in the generator?

Increasing neurons gives the model more capacity to learn complex patterns, which can improve image quality but may increase computation time.

14. What happens when batch size is increased from 64 to 128?

Training becomes more stable with smoother gradient updates, but it requires more memory and may slow down training iterations.

15. What is the effect of adding another layer to the generator?

It can improve the quality of generated images by allowing the model to learn more complex features, but too many layers can make training unstable.

16. What happens when an extra layer is added to the discriminator?

The discriminator becomes stronger and better at detecting fake images, which can make it harder for the generator to learn.

17. Why replace ReLU with LeakyReLU?

LeakyReLU allows a small gradient even when the neuron is inactive, preventing dead neurons and making training more stable.

18. What are the typical loss trends in GAN training?

The discriminator loss usually fluctuates, while the generator loss gradually decreases. Both losses often oscillate due to the competitive nature of GAN training.

19. How does an improved GAN compare to the original?

An improved GAN produces clearer, sharper, and more realistic images with better-defined features.

20. Why does the generator produce noise at the beginning?

At the start, the generator has random weights and has not learned any patterns, so it produces random noise instead of meaningful images.

21. What happens if the discriminator becomes too strong?

The generator receives very small gradients and cannot learn effectively, causing training to slow down or stop.

22. Why is GAN training unstable?

Because two networks are trained simultaneously in competition, small imbalances can cause oscillations, divergence, or mode collapse.

23. What are some real-world applications of GANs?

GANs are used in deepfake video generation, medical image synthesis, super-resolution, art generation, and data augmentation.

24. How can a GAN be modified to generate color images?

The generator output must be changed to produce three channels for RGB images. The dataset should also contain color images, and the architecture should be adjusted accordingly.